

Blade Templates & Directives

Laravel uses a templating engine called Blade. Blade is a simple, yet powerful templating engine that allows you to write clean and concise templates. Blade templates are compiled into plain PHP code and cached until they are modified, meaning they are extremely fast. We can have dynamic content in our views by using Blade directives, which are special tags that start with `@`.

Creating a Blade Template

Let's take the `jobs/index.php` view we created in the previous chapter and convert it to a Blade template. To do this, rename the `jobs/index.php` file to `jobs/index.blade.php`. It will still work as you can use plain PHP in Blade templates. However, Blade provides additional features that make it easier to work with.

Outputting Variables

To output a variable in a Blade template, you can use double curly braces `{{ $variable }}`. Let's update our `jobs/index.blade.php` file to output the title using Blade syntax:

```
<h1>{{ $title }}</h1>
```

Now, when you visit `/jobs` in your browser, you should see the `Available Jobs` heading.

Directives

Blade provides a variety of directives that make it easy to work with data. Directives are prefixed with the `@` symbol. Let's look at a few of the most common directives.

@foreach Directive

Blade provides a `@foreach` directive that makes it easy to iterate over arrays. Let's update our `jobs/index.blade.php` file to display the list of jobs:

```
<ul>
  @foreach($jobs as $job)
    <li>{{ $job }}</li>
  @endforeach
</ul>
```

How much cleaner looking is that? Blade templates are a great way to keep your views clean and concise.

@if Directive

Blade also provides an `@if` directive that allows you to conditionally display content. Let's update our view to only display the list of jobs if there are jobs available:

```
@if(!empty($jobs))
<ul>
  @foreach($jobs as $job)
    <li>{{ $job }}</li>
  @endforeach
</ul>
@endif
```

Now if you clear the array in the route, the list of jobs will not be displayed. We could also use `count()` instead of `!empty()`.

@else Directive

Blade also provides an `@else` directive that allows you to display content when the condition is false. Let's update our view file to display a message when there are no jobs available:

```
@if(!empty($jobs))
<ul>
  @foreach($jobs as $job)
    <li>{{ $job }}</li>
  @endforeach
</ul>
@else
<p>No jobs available</p>
@endif
```

Now if you clear the array in the route, you will see the message `No jobs available`.

There are other directives that pertain to loops. We will look at some of those in the next lesson.